

Champions Club — Complete Technical Guide

Repository walkthrough and implementation reference · 3 October 2026

This guide describes the code currently in this repository. It follows a request from the browser to the database, then explains each domain, the worker, deployment, tests, and the file layout. It is meant for a developer, reviewer, or hackathon judge who needs to explain both **what** the system does and **how** it does it. `SPEC.md`, `PLAN.md`, `API.md`, `REQUIREMENTS.md`, and `README.md` remain the authoritative detailed references.

1. The system in one minute

Champions Club is a sports-club management application for tennis, padel, badminton, and cricket. Customers can create accounts, buy memberships, hold and book courts, join Friday social play, buy products, see bills and receipts, and use a QR membership card. Staff can handle reception, member lookup, POS, kitchen preparation, inventory, CRM, finance, reports, cash shifts, payroll, and reminders. The owner controls staff roles, pricing, plans, and settings.

It is a **modular monolith**: a single Next.js application owns pages and HTTP APIs; `src/modules` owns business logic; PostgreSQL is the source of truth; and `src/worker.ts` runs separately to process durable jobs. Browser state is never the authority for price, role, stock, capacity, or another customer's identity.

```
Browser / React components
  | fetch + session cookie
Next.js App Router pages and API handlers
  | auth -> authorization -> Zod -> module service
Business modules (transactions, pricing, state transitions)
  | Prisma + selected SQL locks/constraints
PostgreSQL <---- separate durable worker
```

2. Technology stack and why each part exists

Layer	Technology	Job in this repository
Full stack	Next.js 16 App Router, React 19, TypeScript 6	Server rendered pages, client interaction, and route handlers in one app. Strict TypeScript catches contract errors.
UI	Tailwind CSS 4, Radix/shadcn style primitives, CVA, Lucide, Geist	Responsive customer and staff themes, accessible controls, icons, typography.
Forms/data	Zod 4, React Hook Form, TanStack Query	Strict request validation, form state, asynchronous server data and refresh.
Persistence	PostgreSQL, Prisma 7, @prisma/adapters-pg	Relational records, migrations, transactions, indexes, and database enforced invariants.
Authentication	Better Auth	Password accounts, verification, reset, session cookies, expiry and revocation.
QR/camera	qrcode, @zxing/browser	Opaque member-card QR generation and staff scanning.
Mail	Nodemailer plus local database inbox	Optional SMTP delivery; offline demo uses persistent local delivery.
Payments	Local simulated mode, manual records, optional Razorpay adapter	Explicitly distinguishes a demo payment from staff-recorded or provider-verified money.
Runtime/ops	Node.js 22+, Docker Compose, GitHub Actions	App, PostgreSQL, worker, local setup and automated checks.

Money is stored as **integer paise**, with basis-point discounts (10,000 = 100%). Timestamps represent UTC instants; club-day and weekly policy boundaries use *Asia/Kolkata*. The API documents ISO UTC timestamps and integer INR paise. There is no separate calculated sales tax in the local demo.

3. Start here: repository map

Path	Purpose
src/app/	App Router pages, loading/error views, and HTTP endpoints. Dynamic API paths route operations by area.
src/components/	Customer, staff, POS, kitchen, owner and shared UI. ui/ has styled button/input primitives.
src/lib/	Shared Prisma connection, Better Auth setup, session and role guard, error envelope, API client, display utilities.
src/modules/account/	Own-account queries, profile edits, invoice ownership, QR card issue/revoke/read.
src/modules/membership/	Purchase, renewal/change, eligibility, active-benefit lookup, shared discounts.
src/modules/billing/	Immutable invoice creation, payment allocation, balances, credits, Razorpay intents and verification.
src/modules/bookings/	Court holds, confirm/cancel/check-in, social play, closures, daily quota, waiting offers.
src/modules/shop/	Online/counter orders, atomic stock reservation, fulfillment, returns and movements.
src/modules/clubhouse/	POS bills/tables, kitchen ticket lifecycle, tab eligibility, settlement rules.
src/modules/crm/, enquiries/	Public lead intake, staff notifications, follow-ups, quotes, conversion.
src/modules/operations/	Shared operation idempotency/locks, request dispatch, scoped read queries.
src/modules/administration/, reports/, staff/	Owner actions, cash/HR/payroll, reports/CSV, plan/settings/role edits.
src/modules/mail/, src/worker.ts	Queued mail/reminders and durable background claim/execute loop.
prisma/schema.prisma	Relational models, enums, relations, uniqueness and indexes.
prisma/migrations/	Seven versioned SQL migrations, including guards Prisma schema alone cannot express.
prisma/seed.ts	Initial club settings, sports/courts, plans, catalogue, menu, tables and demo users.
tests/, tests/integration/	Unit rules and real PostgreSQL/HTTP workflows.
public/	Local sport videos and product SVG images; no remote image dependency for core UI.
scripts/, .env.example, compose.yaml, Dockerfile	Local environment/database setup and container operation.
README.md, API.md, PLAN.md, SPEC.md, REQUIREMENTS.md, JUDGING.md, handoff.md	Setup, contracts, implementation status, requirements, demo and handoff.

src/generated/prisma/ is generated client code. Read the schema and services instead of editing it. .local/, environment files, build output and local database files are not source artifacts to commit.

4. Authentication: from sign-up to protected request

Better Auth is mounted under `/api/auth/*` by `src/app/api/auth/[...all]/route.ts`; its configuration lives in `src/lib/auth.ts`. Public signup creates a `MEMBER` role and server-generated Champions ID. `role` and `championsId` are not accepted as user-controlled inputs. PostgreSQL uniquely enforces email and Champions ID. Passwords require 10–128 characters; email verification is required before customer purchase. Verification and reset links are queued into durable mail. Reset revokes active sessions. Sessions expire after seven days and refresh daily; rate limits are database-backed.

```
// src/lib/auth.ts – shortened from the actual configuration
user: { additionalFields: {
  role: { type: "string", defaultValue: "MEMBER", input: false },
  championsId: { type: "string", input: false },
} },
databaseHooks: { user: { create: { before: async user => ({
  data: { ...user, role: "MEMBER",
    championsId: `CC-${randomBytes(5).toString("hex").toUpperCase()}` }
}) } } }
```

Protected APIs call `requireUser(request, roles?)`; protected server pages call `pageUser(roles?)`. The guard asks Better Auth to validate the cookie, then re-reads the user from PostgreSQL. This means a role demotion applies immediately even if the old session cookie remains present. Owner role changes revoke the target's sessions. Customer services derive customer ID from the session; staff-only flows may select a customer after role checks.

```
// src/lib/access.ts – actual authorization core
const session = await auth.api.getSession({ headers: request?.headers ?? await headers() });
if (!session) throw new AppError(401, "UNAUTHENTICATED", "Please sign in to continue.");
const user = await db.user.findUnique({ where: { id: session.userId } });
if (roles && !roles.includes(user.role)) throw new AppError(403, "FORBIDDEN", "You do not have permission for this action.");
```

Ownership is checked again on record reads: for example `src/modules/account/queries.ts` scopes an invoice by `userId`, and `src/modules/operations/queries.ts` limits each operational record to its customer or permitted staff role. Member card QR payloads contain an opaque revocable identifier, not identity or payment authority. Reception/cashier lookup resolves that identifier to limited fields.

5. HTTP flow, validation and errors

Public endpoints read limited stored data from `/api/public` and `/api/public/availability`; `/api/enquiries` persists public leads. `/api/me` and sibling routes handle the signed-in account. `/api/operations/[area]` and `/[id]` dispatch operational reads/writes. `/api/staff/*` serves staff and owner workspaces. `/api/payments` manages provider intents and `/api/payments/webhook` handles signed provider events. `API.md` lists every method, input and role.

Mutation route handlers require an exact allowed Origin, session, UUID Idempotency-Key, and Zod-parsed body before they invoke a service. Schemas are `.strict()` so unknown fields are rejected. API success uses `{data: ...}`; application failures use `{error:{code,message,fields?}}`. Statuses distinguish unauthenticated (401), forbidden (403), unowned/missing (404), stale/conflicting (409), validation (422), rate limits (429), and disabled integrations (503). Better Auth has its own response contract.

```
// src/modules/operations/dispatch.ts – request entry pattern
sameOrigin(request);
const actor = await requireUser(request);
const key = keyFrom(request);
const input = await jsonBody(request);
if (area === "booking")
  return id ? actBooking(actor, key, id, bookingActionSchema.parse(input))
    : holdBooking(actor, key, bookingSchema.parse(input));
```

The dispatcher routes to a domain service, rather than implementing booking or billing inside the HTTP handler. Read methods use scoped query functions. This keeps the browser and API as adapters and makes domain transactions independently testable.

6. Database model and transaction safety

prisma/schema.prisma defines identities (User, Session, Account, Verification); configuration and plans; memberships/cards; checkout and billing ledger; courts/social/waiting; shop stock/orders; kitchen; leads; staff/HR; cash; gateway; audit, jobs and mail. Foreign keys, unique keys, indexes and status fields represent durable state. The seven SQL migrations add database rules including court overlap exclusion, daily quota and social capacity protections, ledger/invoice guards and administration/gateway safeguards.

Critical mutations run in Prisma transactions. src/modules/operations/core.ts supplies consistent idempotency: a SHA-256 fingerprint of canonical {kind,input}, an advisory lock per actor/key, lookup of an existing Checkout, a 409 on changed-payload reuse, and saving the result with the business write. Repeating the same request returns the original result. Resource-specific locks supplement that: member advisory locks serialize quota/membership actions; court row locks serialize reservation changes; SQL constraints remain the final defense against races.

```
// src/modules/operations/core.ts – essential mechanism
await tx.$executeRaw`SELECT pg_advisory_xact_lock(hashtextextended(${actor.id + ":" + key}, 8))`;
const prior = await tx.checkout.findUnique({ where: { userId_key: { userId: actor.id, key } } });
if (prior) {
  assert(prior.fingerprint === fingerprint, "IDEMPOTENCY_CONFLICT", "This key was already used with different details.");
  return { data: prior.result as T, replayed: true };
}
const result = JSON.parse(JSON.stringify(await work(tx))) as T;
await tx.checkout.create({ data: { userId: actor.id, key, fingerprint, result: result as Prisma.InputJsonValue } });
```

Database rows are source-of-truth even where a browser keeps a draft. The cart and staff POS drafts can survive locally, but a disconnected browser cannot reserve a scarce court/stock item or confirm payment. Financial and membership snapshots remain historical when owner prices later change.

7. Memberships, pricing and QR cards

MembershipPlan stores configurable price, duration, discounts, complimentary weekly courts and Junior eligibility. Membership snapshots the purchased plan economics and term dates. The purchase service in src/modules/membership/service.ts locks both checkout key and member, reads the plan under a database share lock, rejects stale planVersion, checks verification/Junior eligibility and current terms, and atomically records the new term, invoice, allocation, checkout result, audit, card and reminder work. Renewal appends after an unexpired term; plan changes supersede future terms without rewriting history.

```
// src/modules/membership/service.ts – lock and reviewed-price check
await tx.$executeRaw`SELECT pg_advisory_xact_lock(hashtextextended(${userId}, 0))`;
await tx.$queryRaw`SELECT id FROM "MembershipPlan" WHERE id=${input.planId} FOR SHARE`;
if (plan.updatedAt.toISOString() !== input.planVersion)
  throw new AppError(409, "PLAN_CHANGED", "This plan has changed. Close checkout and refresh the page to review the latest price and benefits.");
```

src/modules/membership/pricing.ts is the shared benefit and rounding source for courts, shop and food. Discounted amounts are calculated on the server in integer paise, with one half-up rounding. Booking eligibility uses the session date; shopping/food use the applicable purchase/settlement rules. src/modules/account/cards.ts issues or reuses an opaque QR token; revocation makes an old token useless. A QR identifies the customer to authorized staff; it never settles a bill by itself.

8. Court bookings, social play and waiting lists

`src/modules/bookings/service.ts` validates sport/court, date, opening hour (default 06:00–23:00), 14-day future window, exact one-hour hourly slots, plan benefit, and the two-session club-day limit. The club-day function uses `Asia/Kolkata`; `slot()` turns a local day/hour into a UTC instant. A hold lasts five minutes; confirmation creates/settles the charge through shared billing. Customer cancellation normally needs 12 hours' notice; reasoned staff overrides are explicit. Check-in is single-use. Closures block affected slots.

The database exclusion constraint prevents two active reservations on the same court/time even if concurrent requests race. Member locks and database quota triggers enforce the daily limit across ordinary and social sessions. Friday social play reserves its court once, then separate participants take capacity-limited places. A unique participant rule prevents double joins. FIFO waiting offers are generated after release and expire after the configured window; offer acceptance rechecks quota and availability.

Typical flow: GET availability -> POST booking to hold -> pay/confirm -> one immutable court invoice -> optional check-in or cancellation. Staff reception uses the same service for phone callers/walk-ins, with role-controlled customer or guest selection. This prevents a separate staff code path from bypassing booking rules.

9. Shop and inventory

`src/modules/shop/service.ts` handles the 36-product seeded catalogue with variants. Customer and counter orders share `ProductVariant` stock. Checkout locks/reserves all requested variants together, rejects an insufficient item without leaving a partial reservation, snapshots price/discount into order lines and invoice, then records fulfillment transitions. Pickup stays reserved until collection; delivery consumes stock at dispatch. Unfulfilled cancellation releases reservations. Returns use recorded stock movements; usable returns may restock, damaged returns do not. Partial credits use the historic line price and are bounded so repeated returns cannot over-credit.

The customer cart is a safe local draft. The server validates variant, quantity, current stock, delivery details, membership benefit and total when the order is actually created. Staff inventory screens expose low stock, arrivals/corrections and movement history; corrections cannot consume already reserved stock.

10. Clubhouse POS, kitchen and tabs

`src/modules/clubhouse/service.ts` supports dining tables, member/guest bills, menu items, item notes, additions, cancellations, kitchen tickets and settlement. A waiter/cashier opens an order, adds items and submits a ticket; kitchen moves each ticket through accepted, cooking and ready; cashier marks served. Kitchen preparation and financial payment are separate state machines. Additions make a new ticket revision and charge rather than rewriting the prior item snapshot. A version field rejects stale concurrent kitchen updates.

Membership food discounts apply to final billed prices. Eligible members may keep a tab within the configured ₹5,000 limit and due date; overdue charges block further credit. Partial payments allocate to invoices. A table only closes after remaining tickets are served/cancelled and invoices settle. Alcohol age checks use a known member birth date or a staff-attested guest check; deployment must supply real licensing/policy decisions.

11. Shared billing and payment trust levels

`src/modules/billing/service.ts` owns itemized invoices, payments, allocations, balances, credits and refunds. Each origin generates its charge once. `Invoice/InvoiceLine` snapshot customer and amounts; migrations stop edits to issued financial history. Allocations and credits are bounded under invoice locks. A statement combines departments while retaining origin identifiers. Reports deliberately distinguish invoiced sales, collected money, credits/refunds and outstanding balance.

There are three trust categories: `LOCAL_SIMULATED` is a clearly labelled demo with no funds moved; `MANUAL_RECORDED` is staff-entered cash/card/UPI evidence, not a gateway confirmation; `GATEWAY_VERIFIED` is created only inside the server's

verified provider flow. `PAYMENT_MODE=local` enables the offline demo. Other unconfigured online modes disable purchase. The optional Razorpay adapter in `src/modules/billing/gateway.ts` creates server-priced orders, verifies callback/raw webhook signatures, fetches captured status, INR currency, amount and order ID, then completes through shared billing. Webhooks and recovery jobs tolerate duplicate delivery.

A captured provider payment whose original hold expired or changed becomes `NEEDS_REVIEW` and an unallocated collection. The system does not re-grab a court or item after expiry. An owner can record a provider-verified full exceptional repayment. Provider dashboard action is external; no live/sandbox money movement is claimed by this repository's test results.

12. Leads, staff, owner and reports

Public enquiries (`src/modules/enquiries/service.ts`) save a lead and front-desk notification in one transaction. `src/modules/crm/service.ts` adds assignment, activity notes, durable follow-up, seven-day quote snapshots, status changes and conversion to an account while retaining lead history. A quote estimates; membership processing rechecks the current plan and policy.

`src/modules/staff/service.ts` edits owner-controlled plans/settings and assigns roles; role changes revoke sessions. `src/modules/administration/service.ts` handles business quotes/invoices, employee salary settings, shifts, leave decisions, finalized payslip snapshots, cash opening/payout/closing and mail/gateway review. Cash expected amount is opening float + linked cash receipts - actual linked refunds - payouts; differences require explanation. Finalized payslips and closed shifts are immutable. Payroll withholding is configurable, not statutory filing or bank disbursement.

`src/modules/reports/service.ts` computes Today/week/month/custom ranges using inclusive club dates, department/method drill-downs, operational alerts, utilization and CSV. Sales use invoice issue time; collections use receipt time; credits/refunds have their own event dates; outstanding includes older invoices still open through the range end. CSV export escapes spreadsheet formula prefixes. Staff home filters actionable counts and links by role.

13. Durable worker, mail and reminders

Services write `Job` rows alongside the business change in the same transaction. `src/worker.ts` runs a separate loop, synchronizes reminders about once per minute, and calls `processNextJob()`. The worker claims due jobs with `FOR UPDATE SKIP LOCKED`; a five-minute stale lease can be reclaimed after a crash. Jobs include mail, membership reminders, hold expiry, waiting offers, CRM follow-ups and provider capture recovery. Dedupe keys and state checks make retries safe.

```
-- src/modules/mail/worker.ts - claim shape
UPDATE "Job" SET status='RUNNING', "lockedAt"=NOW(), attempts=attempts+1
WHERE id = (SELECT id FROM "Job" WHERE
  (status='PENDING' AND "runAt"<=NOW()) OR
  (status='RUNNING' AND "lockedAt"<NOW()-INTERVAL '5 minutes')
ORDER BY "runAt" FOR UPDATE SKIP LOCKED LIMIT 1)
RETURNING *;
```

Expiry reminders default to seven days, one day and expiry day at 09:00 club time. Renewals suppress obsolete reminders; dispatch locks the member and rechecks the newest term. `EMAIL_MODE=local` marks a PostgreSQL inbox message delivered without sending externally. `EMAIL_MODE=smtp` sends with Nodemailer and records attempts/errors. SMTP can duplicate delivery if the provider accepted a message just before the worker crashed and its database completion was not saved; the stable Message-ID helps but cannot promise exactly once at the provider.

14. Frontend route and component map

Customer routes: / is the four-sport landing page; /book displays availability and booking; /memberships shows plans/purchase; /shop, /shop/[id], /shop/cart cover retail; /clubhouse shows the menu; /account shows profile, QR, history and ledger; /account/receipts/[id] prints an owned receipt. /login, /signup, /forgot-password, /reset-password use Better Auth client forms. src/components/landing-hero.tsx, courts-view.tsx, membership-options.tsx, shop-view.tsx, cart.tsx, account.tsx and gateway-checkout.tsx contain the main customer interactions.

Staff routes start at /staff; staff-desk.tsx, staff-operations.tsx, operations-ui.tsx, settlement.tsx, and owner-workspaces.tsx render role-specific reception, POS, kitchen, inventory, CRM, billing and owner work. Printable quote/payslip pages live under /staff/quotes/[id] and /staff/payslips/[id]. src/app/globals.css provides black/white/orange customer styling and grey/white/slate staff styling. Local videos/SVG art are in public/. Forms have labels and visible pending/error states; narrow layouts and reduced-motion behavior are part of the UI design.

The frontend uses server data rather than hardcoded operational records. It may keep draft cart/POS inputs locally, then refreshes authoritative responses after mutations. The public read API deliberately omits private reservation/customer details.

15. Setup, runtime and verification

Follow README.md and .env.example for exact setup. At minimum use Node.js 22+, PostgreSQL, configured DATABASE_URL, BETTER_AUTH_SECRET, and BETTER_AUTH_URL. For the no-external-service demo, set PAYMENT_MODE=local and EMAIL_MODE=local; migrate, seed, run the app and run the separate worker. compose.yaml provides local container services; local setup scripts are in scripts/. Never commit .env, generated Prisma client, build output or local database files.

```
npm ci
npm run db:generate
npm run db:migrate
npm run db:seed
npm run dev           # one terminal
npm run worker        # another terminal

npm run lint
npm run typecheck
npm test
npm run test:integration # real PostgreSQL and running app
npm run build
```

The README records the most recent completed validation: ESLint, typecheck, four unit tests, 43 real-PostgreSQL/HTTP integration tests and production build passed on 3 October 2026. Integration coverage includes concurrent court/stock/capacity, idempotency, ownership, refunds, kitchen states, report records, cash/HR invariants, reminders and mocked gateway verification. Those are repository-recorded results, not a fresh run made for this document. Docker execution, real SMTP/Razorpay acceptance, physical camera/LAN testing and remote CI remain deployment checks. The local tests simulate payments; they do not collect real funds.

16. Useful code-reading path and glossary

To understand one write end to end, start at src/app/api/operations/[area]/route.ts, move to src/modules/operations/dispatch.ts, then to the selected domain service, src/modules/operations/core.ts, billing service and prisma/schema.prisma/migrations. To understand a page, follow the route in src/app, its component in src/components, its fetch to API.md, then the module query/service. For payment trust, read src/modules/billing/gateway.ts together with gateway-context.ts and the webhook route. For a delayed task, follow the service's job() call into src/modules/mail/worker.ts.

Hold: a short-lived reservation for a scarce resource before checkout. **Idempotency key:** a UUID that safely repeats a mutation without duplicating its effect. **Checkout fingerprint:** hash of operation and canonical request that rejects

changed data under the same key. **Allocation**: amount of a payment applied to an invoice. **Credit**: amount reducing an invoice for return/cancellation/adjustment. **Snapshot**: historical values stored at transaction time so later settings do not change old records. **Club day**: date interpreted in Asia/Kolkata despite UTC storage. **Durable job**: database record the worker can retry after process failure. **Opaque QR**: random identifier that exposes no personal information in the QR itself.

17. Source references

Primary code: `src/lib/auth.ts`, `src/lib/access.ts`, `src/modules/operations/core.ts`, `src/modules/operations/dispatch.ts`, `src/modules/membership/service.ts`, `src/modules/billing/service.ts`, `src/modules/billing/gateway.ts`, `src/modules/bookings/service.ts`, `src/modules/shop/service.ts`, `src/modules/clubhouse/service.ts`, `src/modules/mail/worker.ts`, `src/modules/reports/service.ts`, `prisma/schema.prisma`, and `prisma/migrations/`. Full API bodies and permission matrix: `API.md`. Requirements and implementation status: `SPEC.md`, `PLAN.md`, `REQUIREMENTS.md`. Local operation and demo evidence: `README.md`, `JUDGING.md`.

Part II — Small-step implementation walkthrough

The first part gives the system map. This part deliberately slows down. Each subsection answers one question, names the exact file to inspect, describes the database effect, and identifies the rule that prevents an incorrect result. Follow the steps in order if you are new to this codebase.

18. Request lifecycle: one operation at a time

18.1 What happens when the user opens the site?

The request reaches a page in `src/app`. The root layout (`src/app/layout.tsx`) supplies the shared shell and providers. The route's server component may fetch persisted public or account data, then passes it to an interactive component in `src/components`. `src/app/loading.tsx`, `error.tsx` and `not-found.tsx` define common fallback states. Public pages have customer styling; `/staff` renders the role-appropriate staff workspace. The server never relies on browser-local drafts as database truth.

18.2 What happens when a button sends a mutation?

A client component calls the typed helper in `src/lib/api-client.ts` or a feature-specific fetch. The browser includes the session cookie and its Origin. The mutation supplies a UUID Idempotency-Key. The Next.js route handler parses route parameters and calls a module. For generalized operations, the thin route calls `src/modules/operations/dispatch.ts`. The dispatcher checks origin and session, parses JSON, runs the matching strict Zod schema and calls the domain service. The domain service does state checks, locks and one transaction. Only after it commits does the response describe a confirmed change.

18.3 Why check Origin if there is a session cookie?

The cookie authenticates the account, but a browser might send that cookie with a request initiated by an unrelated site. `sameOrigin()` in `src/lib/errors.ts` requires the request's Origin to exactly match the configured trusted origin list. This check is for application mutations. Better Auth has its own trusted-origin configuration. There is no permissive cross-origin API header. An absent or untrusted Origin produces HTTP 403.

18.4 How are malformed requests handled?

`jsonBody()` reads the raw request, rejects a body over 16,384 characters, then parses JSON. Zod schemas reject missing, invalid or extra fields. `route()` catches known application errors and Zod errors and returns a stable JSON envelope. It hides unexpected internal details behind a generic 500 while logging only the error class/name. The response is marked `Cache-Control: private, no-store` to prevent a protected response from being cached as shared content.

18.5 Why is there a `Checkout` table for unrelated operations?

The table stores a caller/key, fingerprint and result for repeat-safe writes. It is a general idempotency ledger, not just a visual shopping checkout. `operation()` wraps many bookings, shop, POS, CRM and staff changes. If the network loses a response after the commit, retrying with the same key returns the saved result. If the payload changed under the same key, it returns conflict. The unique `(userId, key)` constraint and per-key advisory lock stop two simultaneous retries from duplicating a write.

18.6 What is the difference between application locks and SQL constraints?

A service lock orders competing transactions so it can calculate a correct decision from current rows. A SQL constraint or trigger protects the invariant even if a future code path forgets the lock. Examples: `courtLock()` locks a court row, while the reservation exclusion constraint rejects overlapping active time ranges. Member advisory locks serialize a member's quota checks, while the quota trigger guards direct SQL writes. Both layers matter because multiple app and worker processes may write concurrently.

19. Accounts and identity in tiny steps

19.1 What exactly is stored for an account?

`User` stores name, unique email, verified flag, database role, unique Champions ID, optional phone and birth date. Better Auth also uses `Session`, `Account`, `Verification` and `RateLimit`. `Session` links a cookie token to a user and expiry; `Account` stores credential/provider information; `Verification` stores temporary email/reset verification data; `RateLimit` stores request counters. An account exists before the person buys a membership. Staff roles are values on the same `User` model, not separate login systems.

19.2 How does public signup avoid owner or cashier creation?

`src/lib/auth.ts` declares `role` and `championsId` with `input:false`. Its user-create hook overrides the role to `MEMBER` and generates `cc-` plus random hex. It also trims and bounds the name. The database uniquely enforces the email and Champions ID. An attacker who adds `role:"OWNER"` to signup cannot make the hook assign that privilege. Existing roles can only be changed through the owner-controlled staff path.

19.3 What does email verification change?

Better Auth requires verification for password accounts and queues a one-hour verification link through `queueMail()`. In local mode the email appears in the owner-only test inbox after the worker processes it; SMTP mode sends externally if configured. The membership service independently rejects an unverified customer purchase. A successful verification changes the persisted `emailVerified` status. The link's secret is not put into audit records or normal account API responses.

19.4 How do login, logout and reset work?

Better Auth creates/validates session cookies on sign-in and removes or revokes them on sign-out. Sessions expire after seven days. The reset flow queues a one-hour link; resetting a password revokes sessions. `src/components/auth-form.tsx` and `reset-form.tsx` implement browser forms, while `src/lib/auth-client.ts` provides the client. The server does not trust a client-side "logged in" flag: every protected request asks Better Auth to resolve the session.

19.5 How is a role enforced after it changes?

`requireUser()` reads `session.user.id`, then fetches the current user row from PostgreSQL and checks the role list. Owner role assignment in `src/modules/staff/service.ts` also revokes the target's sessions. Even before a new login, the fresh database role check prevents old privileges from being used. Staff-specific record services additionally check whether that role may access the requested department or action.

19.6 How is another customer's data protected?

The `/api/me` family never accepts a customer ID to override the session user. `ownedInvoice(userId,id)` includes both values in the query. Operational record queries use the actor ID or explicit staff privileges. A guessed receipt URL is not enough: the page and API recheck ownership. In staff-created walk-ins, `subject()` permits a staff-selected account/guest; for MEMBER it forces `userId=actor.id` and rejects selection of another member.

19.7 What does the QR represent?

`MemberCard` holds one unique, randomly generated token per user and a revocation timestamp. `readCard()` renders `champions:card:<token>` as a PNG data URL with `qrcode`, but only when the membership is active. The token carries no name, email or discount value. `issueCard()` reuses an active card and rotates a revoked one under the member lock. `revokeCard()` timestamps it and audits the action. Staff lookup resolves a live token to limited current benefits; the QR itself never authorizes billing.

20. Membership purchase in ten steps

20.1 The member sees current plans

`/memberships` obtains active plans from the public read service and displays each plan's price, duration, discounts and free sessions. This public response contains `updatedAt`, exposed to the purchase form as `planVersion`. The client displays a price for review, but the server does not accept that displayed price as payment authority.

20.2 The member submits a strict command

The purchase body contains `planId`, `action` (purchase, renew, change), `acceptPolicy:true`, and an ISO `planVersion`. It has no amount, discount, role or customer ID. The route obtains user ID from the session and a UUID idempotency key from the header. `Zod.strict()` rejects unexpected fields.

20.3 The service serializes retries and term changes

`purchaseMembership()` first takes an advisory lock for checkout actor/key, then one for the target member. The first lock ensures duplicate clicks share one result. The member lock serializes purchase, renewal, plan change and birthday updates. A previous Checkout result returns before later plan checks, so a true retry can still succeed after a price edit.

20.4 The server verifies the reviewed plan

The service locks the selected plan row for sharing, loads active terms, checks verified email for a customer purchase, checks plan activity and compares `plan.updatedAt` to `planVersion`. If the owner changed price/benefits after the member saw the page, it returns `PLAN_CHANGED` without creating records. The person must refresh and approve the new economics. Junior eligibility is checked using birth date at the term start.

20.5 The service calculates the next term

purchase requires no existing live term. A renewal of the same plan appends to the end of an unexpired term. A plan change begins immediately and supersedes future terms rather than editing them in place; it does not prorate. This preserves a history of exactly which plan was sold and when it applied.

20.6 The price and benefits are snapshotted

The term stores `pricePaise` and `planSnapshot` as purchased. If an owner later changes Gold from ₹12,000 to another price or adjusts discounts, that earlier membership and its invoice do not silently change. `Downstream benefits()` reads the active term's snapshot, not the current editable plan, at the relevant date.

20.7 The invoice and payment are recorded

`issueMembershipInvoice()` calls shared `issueInvoice()` and `payInvoice()` within the same transaction. The invoice records one historical membership line and origin ID. In local mode the payment source is `LOCAL_SIMULATED`; staff cash/card/UPI is `MANUAL_RECORDED`; a genuine provider capture uses `GATEWAY_VERIFIED`. The payment allocation points to the invoice.

20.8 The card, audit and reminders follow

The purchase flow issues/reuses a card, writes a sensitive-action audit entry, and schedules expiry reminders/job records transactionally. If any required database step fails, the transaction rolls back together. Reminders later recheck the newest term so a renewal can suppress old notices.

20.9 The result is saved for exact replay

The Checkout record stores the fingerprint and stable result containing membership/invoice IDs and term dates. A repeated identical command returns `replayed:true`; the same key with altered details returns 409. Concurrent duplicate requests therefore cannot buy two overlapping terms or create two invoices.

20.10 The account page reads the result

`src/modules/account/queries.ts` loads the signed-in profile, membership terms, invoices and other history. The account UI shows the active plan/card, renewal prompt if expired, or membership invitation if never purchased. It can display issued receipt data without recomputing its price from current settings.

21. Booking one court in small steps

21.1 Public availability is deliberately limited

`/api/public/availability` receives sport and club date, reads persisted courts/reservations/closures, and returns available or elapsed hourly slots with court names and guest prices. It does not reveal who reserved a court. The UI uses this as a preview; the service rechecks everything on hold because another user may book the slot meanwhile.

21.2 Input becomes a UTC slot

`bookingSchema` validates court, day, hour and optional trial request. `slot(day, hour)` constructs an instant at +05:30; a one-hour end is derived from that start. `clubDay()` formats instants in `Asia/Kolkata` for the two-session daily quota. The database stores actual instants as `timestampz`, so devices in different host timezones still refer to the same session.

21.3 The service checks business policy

`holdBooking()` verifies court activity, opening hours, future window, trial eligibility, closure status, member/guest price and plan benefit at the session date. It also checks existing ordinary reservations and social places in that member's club day. Gold/Junior complimentary weekly benefits use the club week and active holds. Settings come from `ClubSettings`, not hardcoded browser fields.

21.4 The court and member are protected against races

The service locks the court and member before it creates an active hold. If two users target the final court hour, one succeeds and the other sees a conflict. The exclusion constraint remains a database backstop. Direct SQL or a future code path still cannot store overlapping active reservations, and quota triggers protect the member limit.

21.5 Hold, invoice and expiry work are persisted

An accepted hold gets `HOLD` status, a server-computed `pricePaise` and `priceSnapshot`, a `holdUntil`, and a durable expiry job. The invoice is linked to the reservation through `invoiceId`. The job can expire an abandoned hold and free capacity after the server-controlled five minutes. A browser timer is only a display aid.

21.6 Confirm, cancel and check-in are separate actions

Confirming uses the same checkout/payment trust rules as other departments. Cancelling applies the notice policy, credits as needed and releases the court; staff override requires a reason. Reception check-in is limited to the authorized window and records `checkedInAt` once. Replaying any action with its idempotency key returns its prior result rather than duplicating payment or check-in.

21.7 How does a waitlist offer get made?

If a slot is full, a customer may create a `waitlistEntry` for court/start. Release or expiry schedules `OFFER_WAITLIST`. The worker selects the earliest eligible entry, rechecks capacity/quota, and creates an offer that expires after the configured period. Ineligible people are skipped. Acceptance still goes through the normal booking confirmation path; receiving an offer does not guarantee a bypass of current rules.

22. Social play, stock and kitchen as separate state machines

22.1 Why is social play not twelve court reservations?

`SocialEvent` points to one `Reservation` for the court/hour. `SocialParticipant` stores each member's place, price snapshot, hold/confirmation and invoice. This matches the real resource: one court is occupied once, while capacity belongs to the event. The `(eventId,userId)` unique key and capacity trigger prevent duplicate or over-capacity joins. A social place counts toward the two-session club-day allowance.

22.2 What happens when a social place is cancelled?

The participant state changes without releasing the underlying court if the event continues. Its charge can be credited, and the next waiting customer can be offered that place. Cancelling the entire event is a separate reasoned staff action; it releases the one court reservation and handles participant invoices. This distinction avoids accidentally making the court publicly available while an event remains active.

22.3 How is stock represented?

`Product` describes a catalogue item. `ProductVariant` is the sellable SKU with integer `stock`, `reserved`, `pricePaise` and unique `sku`. `ShopOrderLine` snapshots name, label, quantity, unit price, discount, total and returned count. `StockMovement` records every arrival/correction/dispatch/return delta with an origin key. Product detail and cart pages read current variants, but order creation re-reads them under transaction protection.

22.4 Why track both stock and reserved?

Available quantity is stock minus reserved. An online hold increases reserved so the same last unit cannot be sold at the counter. A pickup order keeps the reservation until collection. Dispatch consumes stock and releases the reservation. Cancelling an unfulfilled order only releases its reservation. A return can increase stock if usable; a damaged item is recorded without re-entering saleable stock.

22.5 How does an order avoid partial success?

`holdOrder()` validates all cart lines and locks/updates the shared variants in a transaction. If the last SKU is insufficient, the entire order and its other reservations roll back. The server recalculates member discount and delivery fee; client totals are ignored. Order lines and invoice preserve the original prices. Fulfillment actions check the current order state so a repeated dispatch or return cannot double-move stock.

22.6 What is a POS bill versus a kitchen ticket?

`KitchenOrder` is the table/member/guest bill and overall lifecycle. Each `KitchenTicket` is a submission revision to the kitchen; `KitchenLine` snapshots a menu item, quantity, price, discount, note and status. Adding items to an open table creates another ticket rather than mutating the old ticket's original contents. The bill may be unpaid while food is ready, or paid while preparation still runs: payment and preparation are independent.

22.7 Why does a kitchen ticket have a version?

Two kitchen staff may view the same incoming ticket. An action includes the version the browser saw. `prepareTicket()` compares it to the current row and rejects stale updates, then increments it. Without that check, a late “accepted” click could overwrite another staff member’s “ready” update. State transitions are also limited to the proper role, so the cashier serves while the kitchen prepares.

22.8 When is a table allowed to close?

The service checks that remaining tickets are served/cancelled and the linked invoices have no outstanding balance. A member tab can defer payment only within the configured limit and if overdue rules permit. Partial payments are real ledger allocations rather than a client-side “paid” flag. This allows a cashier to reconcile the exact open amount at any time.

23. Billing math with an example

23.1 How does one invoice get its amount?

`issueInvoice()` accepts server-built lines. For each line, subtotal is $\text{quantity} \times \text{unitPaise}$; invoice subtotal sums those amounts. Discount paise sums line discounts. Total is subtotal minus discounts. Invoice stores department, origin, customer snapshot, issue/due dates and unique invoice number. A unique $(\text{department}, \text{originId})$ stops the same booking/order/member term from creating duplicate charges.

23.2 Example: ₹800 court with a 25% benefit

The original price is 80,000 paise. A 25% discount is 2,500 basis points, which produces a 20,000-paise discount and a 60,000-paise invoice total (₹600). The basis-point function uses integer arithmetic and half-up rounding once. The invoice stores the final line values, so a future plan change cannot turn that old ₹600 charge into a different amount.

```
// Actual shared rounding function in src/modules/billing/service.ts
export function discountedAmount(paise: number, basisPoints: number) {
  return paise - Math.floor((paise * basisPoints + 5000) / 10000);
}
```

23.3 How is outstanding balance derived?

`balance()` loads invoice allocations and credits. It computes $\text{paid} = \text{sum}(\text{allocations})$, $\text{credited} = \text{sum}(\text{credits})$, then $\text{outstanding} = \max(0, \text{totalPaise} - \text{paid} - \text{credited})$. For a ₹600 invoice with a ₹200 payment and no credit, outstanding is ₹400. A later ₹100 credit makes it ₹300. Historical payments stay visible even when a credit changes what is owed.

23.4 Why lock an invoice during payment?

`payInvoice()` executes `SELECT ... FOR UPDATE` on the invoice, recomputes the balance inside the transaction, and requires a positive integer amount no greater than outstanding. If two cashiers try to settle the last ₹300 simultaneously, one locks first; the second sees the updated balance and cannot overallocate. Database ledger guards reinforce this boundary.

23.5 What makes a payment “gateway verified”?

The public request cannot choose a trusted source by setting `method: "GATEWAY"`. `payInvoice()` demands an internal `verifiedPayment` context with matching user, amount and optional invoice ID. That context is established only after provider signature, captured status, order, amount and currency checks. Cash/card/UPI records require a staff role and are tagged `MANUAL_RECORDED`. Local demo records are tagged `LOCAL_SIMULATED` and require `PAYMENT_MODE=local`.

23.6 How do credits and refunds relate?

`creditInvoice()` is repeat-safe by `originId`, locks the invoice, prevents credits above its total charge and records a reason/audit. A credit reduces what is owed. If money was already collected beyond the new net charge, it creates a refund record. Local simulation marks a simulated refund recorded. Real manual cases remain pending until staff records actual repayment; creating a credit alone does not prove money left the business.

24. Optional Razorpay path in individual steps

24.1 Create an intent

The customer requests `/api/payments` with a kind and target, not a trusted amount. The gateway service rechecks ownership, current hold or invoice, and server price, then creates a provider order and a `GatewayIntent` linked to that request. Intent idempotency prevents repeated browser clicks from creating unrelated checkout attempts.

24.2 Receive a callback or webhook

The browser callback supplies provider payment ID and signature. The webhook sends a raw body and separate webhook signature. The server verifies the HMAC with configured secrets. Neither an unsigned browser success message nor a client-selected method marks an invoice paid. The webhook can enqueue a durable capture job so completion survives a browser closing.

24.3 Fetch the provider truth

`completeIntent()` fetches the provider payment and checks `captured`, order ID, INR currency and exact amount. Only then can it use internal verified context to invoke the same membership/booking/order/social/invoice settlement flow used by local checkout. Duplicate callbacks and jobs resolve to the same result.

24.4 Handle capture after an expired hold

If provider money is captured but the reservation/stock hold is stale, the service records `NEEDS_REVIEW` and an unallocated collection instead of claiming scarce resources again. The owner sees that exception in gateway and financial reports. A provider-side repayment can be verified and recorded once; this code does not initiate an automatic payout. The repository tests use mocked provider responses, not real sandbox/live funds.

25. Worker internals in individual steps

25.1 How is a job created without losing it?

`job(tx, kind, dedupeKey, payload, runAt)` upserts a `Job` in the same transaction as the business event. For example, a booking hold and its expiry job commit together. If the transaction rolls back, neither exists. `dedupeKey` is unique, so retrying a scheduling action does not produce a second logical job.

25.2 How do two workers avoid taking the same job?

The claim SQL selects a due job with `FOR UPDATE SKIP LOCKED`, updates it to `RUNNING`, timestamps its lease and increments attempts. Another worker skips that locked row and can claim different work. A job stuck in `RUNNING` for over five minutes is eligible for recovery, so a crashed process does not permanently strand it.

25.3 What happens if job processing fails?

The worker catches the failure, records a safe error/attempt state, and schedules a retry with backoff or eventually marks failure, according to the job handler. Handlers recheck current business state before acting. Expiring a hold that was already confirmed is a no-op; obsolete reminder delivery is suppressed; repeating a stock-release or gateway completion must not duplicate its financial effect.

25.4 How do reminders know when to send?

`src/modules/mail/reminders.ts` calculates configured offsets (default 7, 1 and 0 days) and hour (09:00) in the club timezone, queues mail/job rows and periodically synchronizes older terms. The outer `src/worker.ts` loop invokes this sync roughly every minute. When the job is about to deliver, it locks the member and compares the message's membership ID to the latest active term. Renewal makes the old message obsolete.

25.5 What is special about local email mode?

In local mode, the worker changes a persisted `MailMessage` to `DELIVERED` without contacting a provider. The owner-only inbox shows the test messages, including verification/reset links. This supports offline judging and durable retry tests. In SMTP mode, Nodemailer sends externally with configured host, port and credentials. The provider may accept a message just before a crash, so SMTP delivery is at least once rather than mathematically exactly once.

26. Owner and staff tasks in individual steps

26.1 Reception desk

Reception reads a daily court calendar and limited member search, identifies someone via email/Champions ID/QR, creates a booking or guest walk-in through the common booking service, handles check-in and may process a reviewed membership. Its extra authority comes from role checks, not from bypassing pricing or capacity. Reasoned overrides are audited.

26.2 CRM desk

Public enquiry intake writes `Lead` and `StaffNotification` in one transaction. Reception/owner can assign a lead, append `LeadActivity`, schedule a durable follow-up, create a time-limited `LeadQuote`, update status or link/convert to an account. The lead record and activity remain after conversion so the history is not erased.

26.3 Cash shift

An authorized cashier/receptionist opens a `CashShift` with an opening float. Cash payments and recorded cash refunds attach to the open shift under a lock. Payouts have reasons. At close, expected cash equals opening + linked receipts - linked refunds - payouts. Counted cash is entered and any discrepancy requires an explanation. The closed shift becomes historical; reports also show unassigned older cash separately.

26.4 Employee and payroll

The owner links an existing staff user to an `Employee`, configures title/salary, schedules shifts, approves/rejects leave, and finalizes a `Payslip` for a period. A payslip stores a snapshot of salary, adjustment and configured withholding at finalization; later settings do not rewrite it. A staff member can see only their own employee/payslip records. The app calculates records and exports; bank transfers and statutory tax filing happen outside it.

26.5 Reports and drill-down

The owner chooses Today, week, month or an inclusive custom club-date range. The report service fetches supporting invoices, payments, allocations, credits, refunds and operations. Revenue measures have distinct clocks: sales use invoice issue date, collections use payment receipt date, credits/refunds their own dates, and outstanding spans earlier invoices still open at period end. Clickable totals drill to records, and CSV preserves paise values while escaping formula-like strings.

26.6 Business quote versus fulfillment

An owner `BusinessQuote` snapshots customer, department, item descriptions, quantities, unit prices, total and validity. Issuing its invoice creates an immutable financial charge once. It does not reserve courts, reduce inventory or activate a membership. Those operational effects must go through the ordinary department service; this distinction prevents a quote/invoice from silently promising scarce resources.

27. Database table catalogue, grouped by ownership

27.1 Identity and configuration tables

User is the person and role. Session, Account, Verification, RateLimit belong to authentication. ClubSettings is one editable policy row. Sport/Court define facilities, and MembershipPlan is editable product configuration. Configuration is read at the time of an operation; purchased history is stored elsewhere in snapshots.

27.2 Financial tables

Checkout records idempotency. Invoice and InvoiceLine record charges. Payment is a money event with source and reference. PaymentAllocation applies a payment to an invoice. Credit reduces an invoice, and Refund tracks repayment caused by a credit. GatewayIntent and GatewayRepayment track provider checkout and exceptional captured-payment handling. AuditLog records actor/action/record/reason without password or session secrets.

27.3 Membership and scheduling tables

Membership is a dated term with price/benefit snapshot. MemberCard is the opaque QR identifier. Reservation is one court/time booking or hold. CourtClosure blocks a time range. SocialEvent is an event on one reservation; SocialParticipant is one person's place. WaitlistEntry tracks waiting/offered status and the resulting reservation/participant link.

27.4 Retail and dining tables

Product and ProductVariant define catalogue/SKU/stock. ShopOrder and ShopOrderLine store order and historical line economics. StockMovement explains each quantity change. MenuItem and DiningTable are POS configuration. KitchenOrder, KitchenTicket and KitchenLine separate a table bill from successive kitchen submissions and their individual preparation states.

27.5 CRM, staff and work queue tables

Lead, LeadActivity, LeadQuote and StaffNotification hold enquiry history and staff attention. Employee, StaffShift, LeaveRequest, Payslip cover staffing/payroll. CashShift/CashPayout cover cash reconciliation. BusinessQuote covers owner quotes. Job and MailMessage are the durable work queue and delivery record.

28. How to explain the project in a technical review

28.1 A short architecture answer

"Next.js serves the React pages and thin API handlers. Better Auth resolves sessions; handlers enforce origin/roles and parse strict Zod inputs. Services in `src/modules` own policy and Prisma transactions. PostgreSQL constraints and explicit locks protect concurrent resources. A separate worker claims durable database jobs for expiry, reminders and recovery. All operational UI reads persisted records."

28.2 A short concurrency answer

"For repeated requests we use a UUID idempotency key, a canonical payload hash, an advisory lock and a saved result. For scarce resources we lock the member/court/SKU and rely on database constraints/triggers as a backstop. A losing concurrent request returns a conflict instead of double-booking or overselling."

28.3 A short security answer

“Public signup can only create a member. Protected requests revalidate the session and current database role. Customer identity is derived from the session and reads are scoped by owner ID. Mutations check exact Origin and strict schemas. The QR contains a revocable opaque ID. Prices and gateway payment status are computed or verified server-side.”

28.4 A short money answer

“Every department creates an itemized immutable invoice through shared billing. Payments are separate events allocated to invoices; credits reduce charges and refunds track actual repayment. The app labels local test money, staff-recorded money and provider-verified money differently. Reports distinguish sales, collections, credits/refunds and outstanding.”

28.5 A short limitations answer

“The full local workflow is implemented and tested against PostgreSQL. SMTP and Razorpay need real credentials and external acceptance testing. Physical QR camera, LAN devices, Docker runtime and remote CI still require deployment checks. Payroll records configurable withholding but does not perform bank disbursement or statutory filing.”